

Week 1

2. Software Design Patterns

Introduction to the Observer Pattern

Dr. Andreas S. Maniatis

Assistant Professor

School of Computer Science

COMP.2800 – Software Development [2026W]

Wednesday, January 7th, 2026 | 14:30 – 15:50 | ER 1118



University of Windsor

Lecture 1: Review



Let's get to know
each other



Getting the best
out of the course:
Best practices



Course Outline



Agenda

- Importance
- Problem Description
- Solution
- Using Observer Patterns
- Implementation





Learning Outcomes



Learning Outcomes

- Understand the purpose of design patterns
- Understand the necessity of dealing with changes
- Describe the context of using the observer pattern
- Learn how to use the observer pattern in Java



Part I

Design patterns



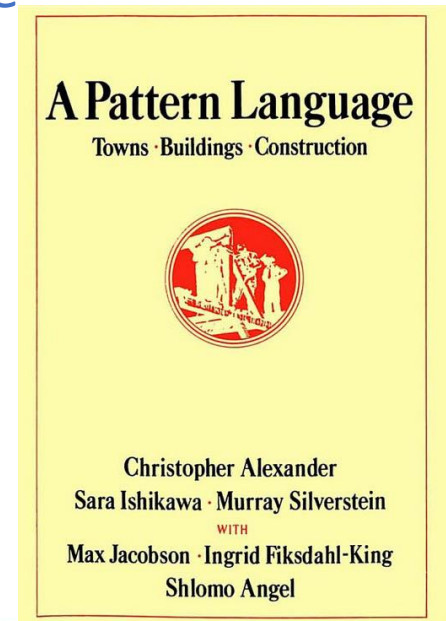
○ Why should we learn design patterns?



Why should we learn design patterns?

“Each pattern describes a problem which occurs over and over again in our environment, and then describes the core of the solution to that problem in such a way that you can use this solution a million times over, without ever doing it the same way twice”

~Christopher Alexander

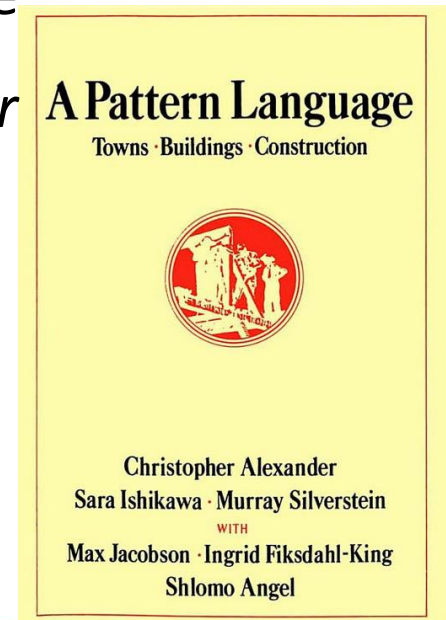


Why should we learn design patterns?

1

“Each pattern **describes a problem which occurs over and over again** in our environment, and then describes the core of the solution to that problem in such a way that you can use this solution a million times over, without ever doing it the same way twice”

~Christopher Alexander



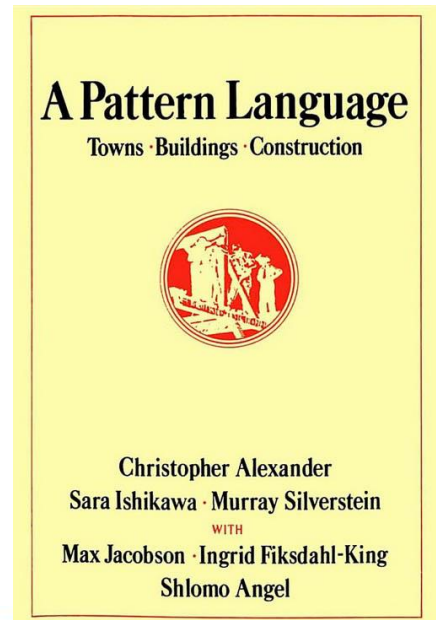
Why should we learn design patterns?

1

“Each pattern **describes a problem which occurs over and over again** in our environment, and then **describes the core of the solution to that problem** in such a way that you can use this solution a million times over, without ever doing it the same way twice”

~Christopher Alexander

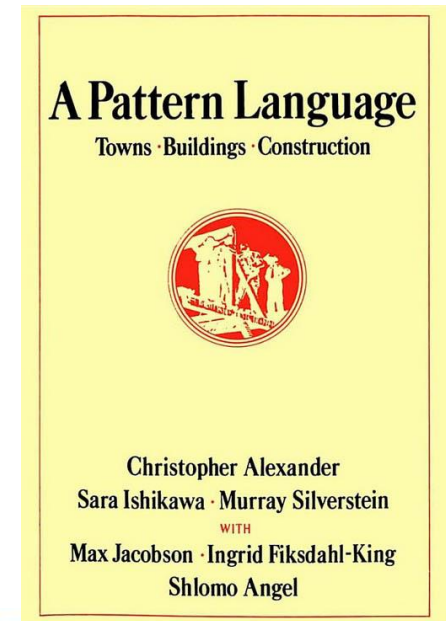
2



Why should we learn design patterns?

1 “Each pattern **describes a problem which occurs over and over again** in our environment, and then **describes the core of the solution to that problem** in such a way that you can **use** 2 **this solution a million times over**, without ever doing it the same way twice” 3

~Christopher Alexander





About the importance of design patterns

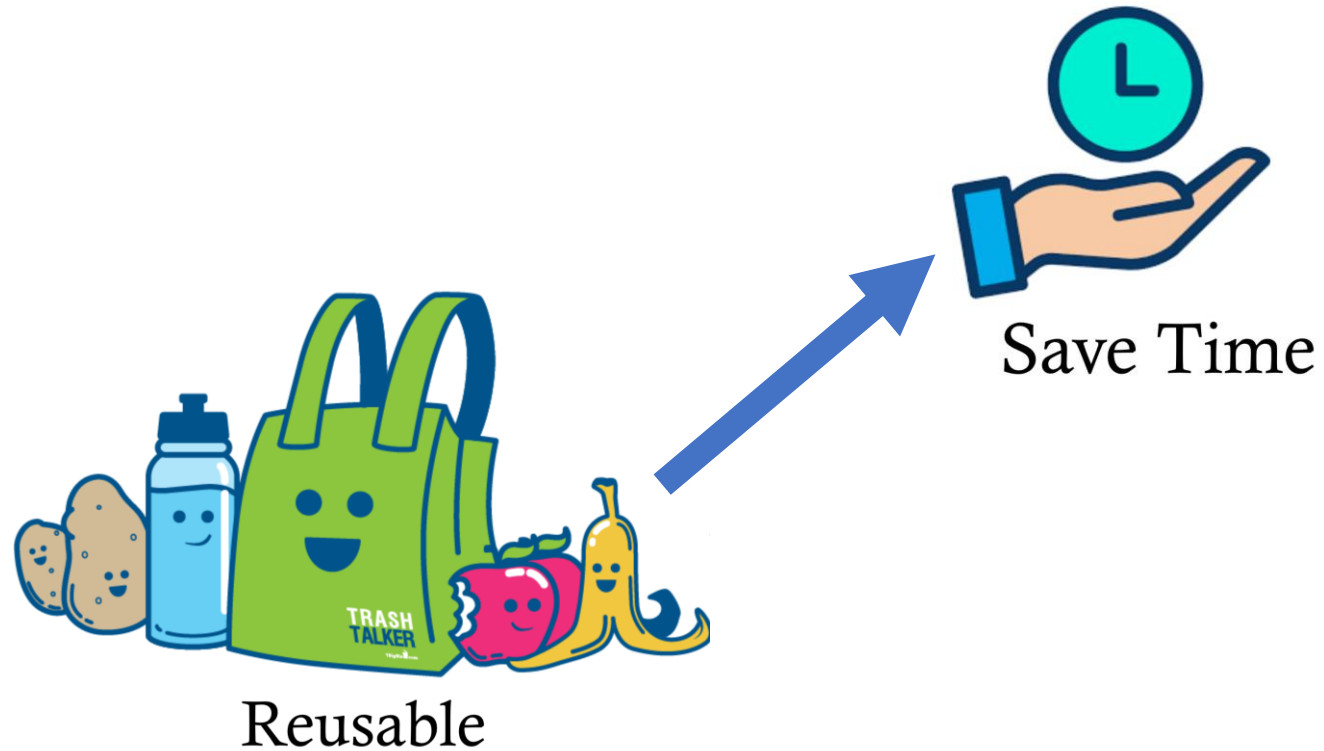


Why are design patterns important?

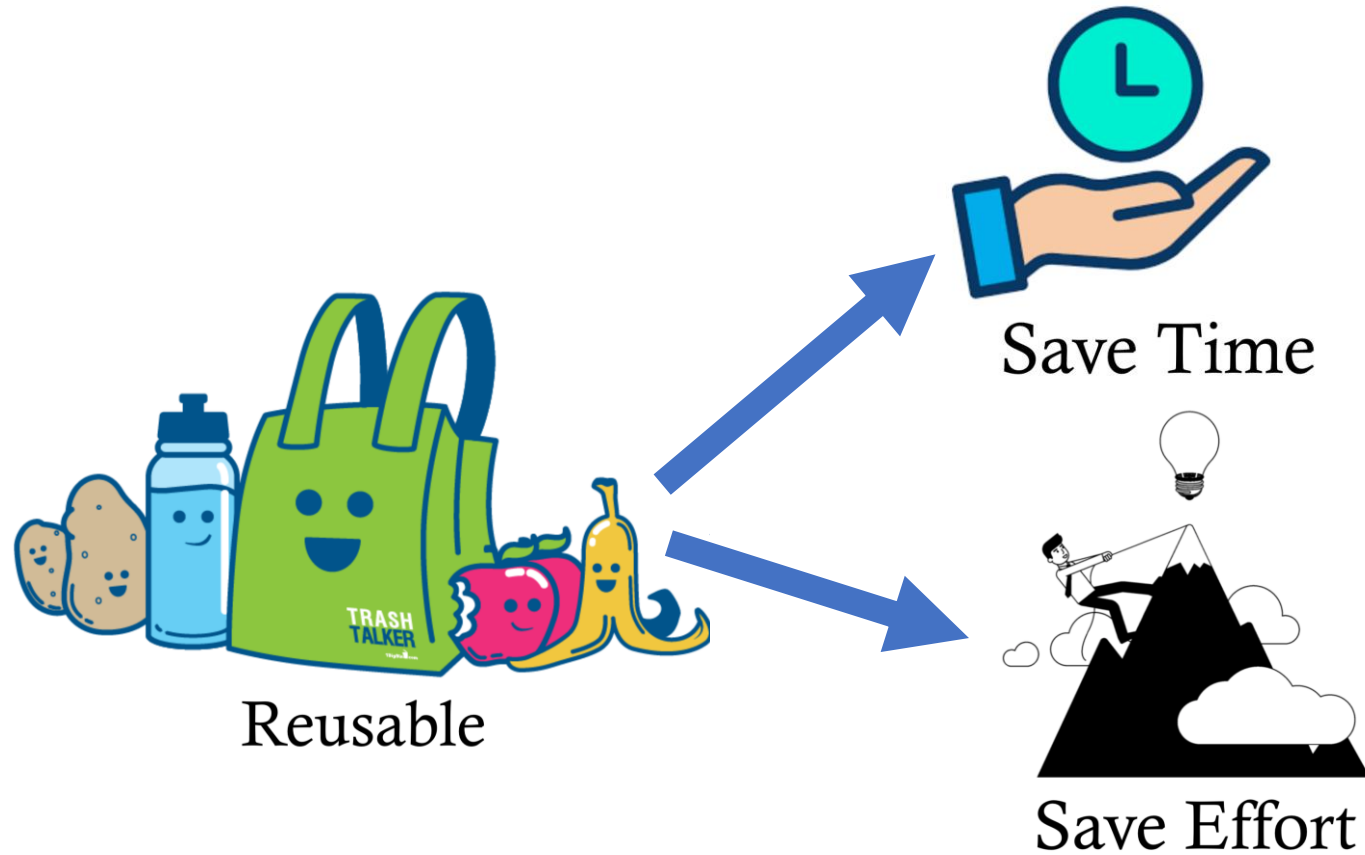


Reusable

Why are design patterns important?



Why are design patterns important?



○ A Catalog of Design Patterns



A Catalog of Design Patterns

• **Creational Patterns**

(abstracting the object-instantiation process)

- Factory Method
- Builder

Abstract Factory
Prototype

Singleton



A Catalog of Design Patterns

• **Creational Patterns**

(abstracting the object-instantiation process)

- Factory Method
- Builder

Abstract Factory
Prototype

Singleton

• **Structural Patterns**

(how objects/classes can be combined)

- Adapter
- Decorator
- Proxy

Bridge
Facade

Composite
Flyweight



A Catalog of Design Patterns

• **Creational Patterns**

(abstracting the object-instantiation process)

- | | | |
|------------------|------------------|-----------|
| • Factory Method | Abstract Factory | Singleton |
| • Builder | Prototype | |

• **Structural Patterns**

(how objects/classes can be combined)

- | | | |
|-------------|--------|-----------|
| • Adapter | Bridge | Composite |
| • Decorator | Facade | Flyweight |
| • Proxy | | |

• **Behavioral Patterns**

(communication between objects)

- | | | |
|-------------------|-------------------------|----------|
| • Command | Interpreter | Iterator |
| • Mediator | Observer | State |
| • Strategy | Chain of Responsibility | Visitor |
| • Template Method | | |

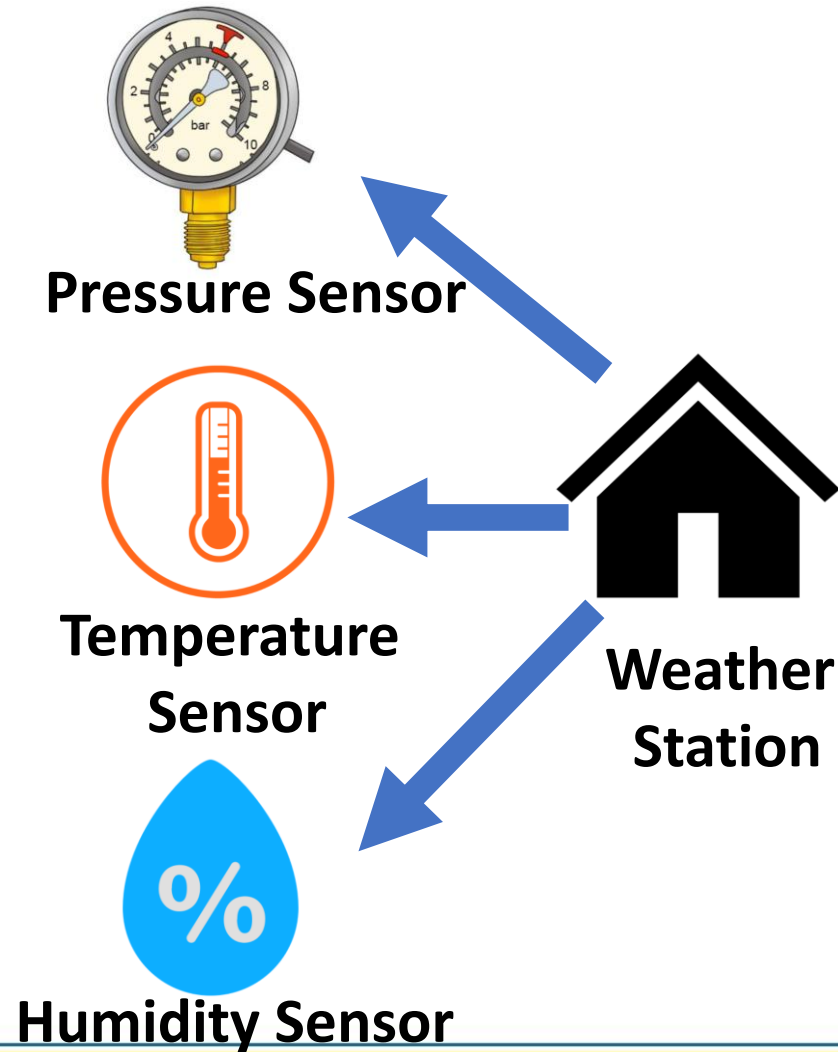


Part II

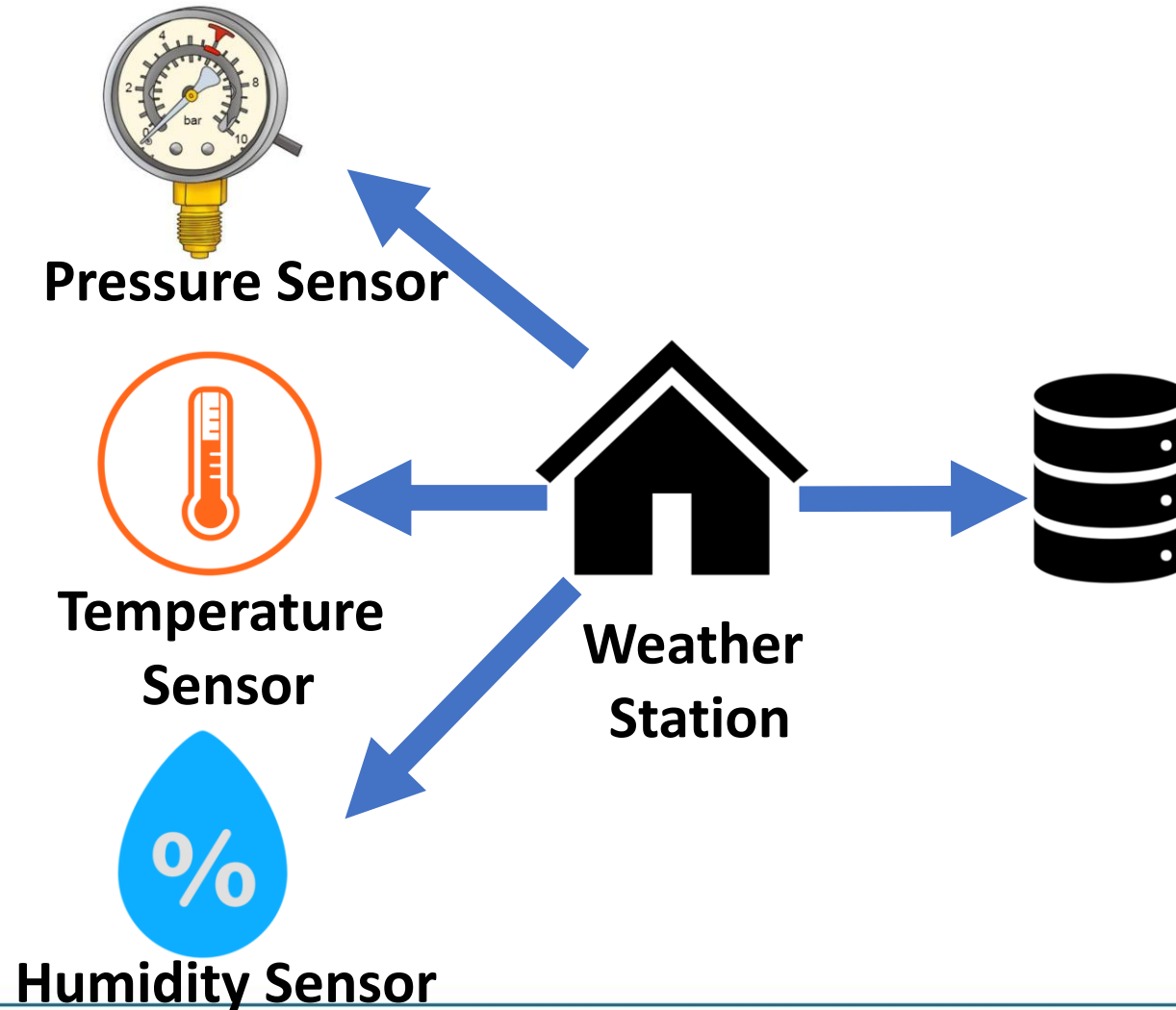
Design Patterns by Example



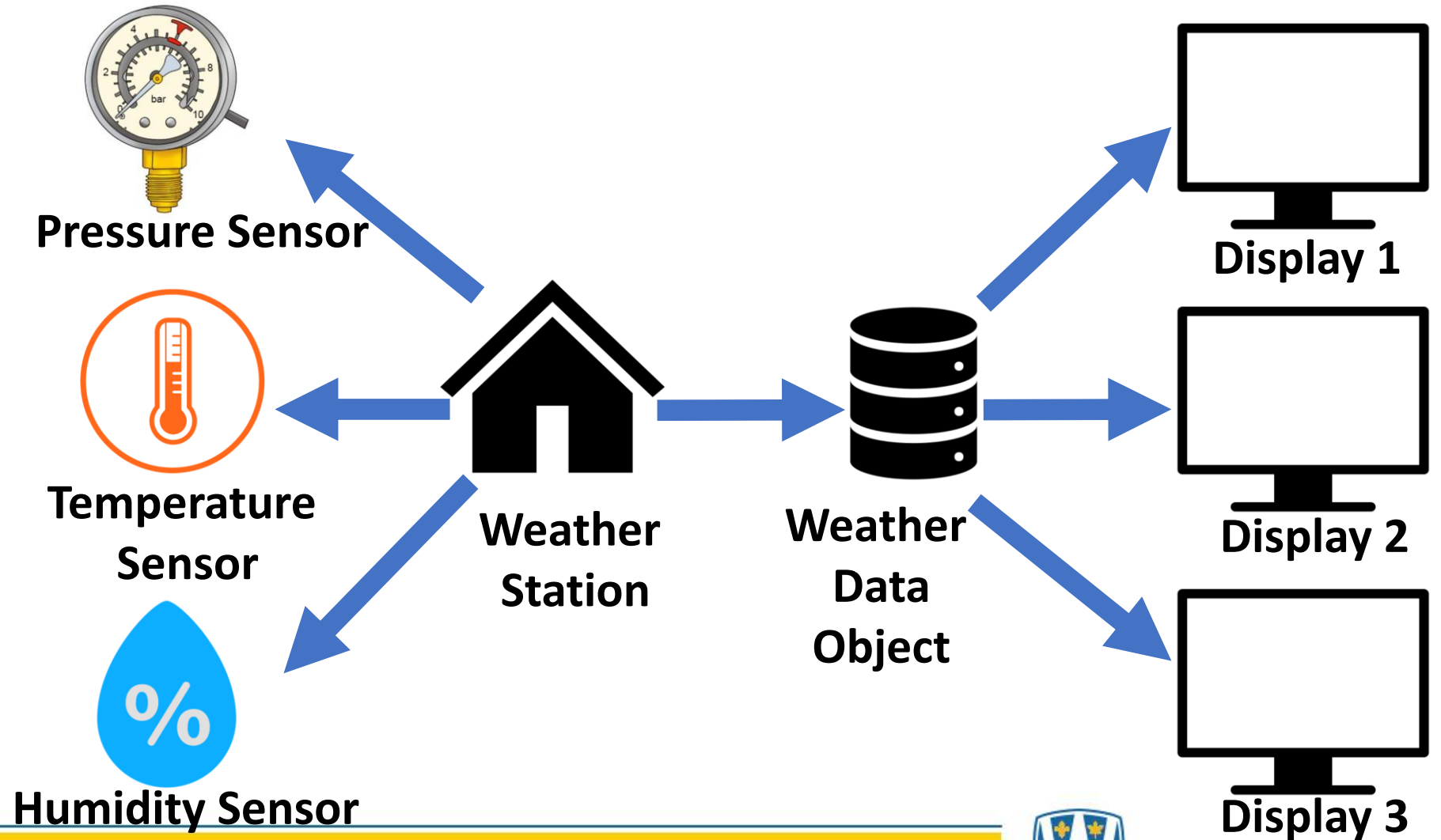
Problem Description



Problem Description



Problem Description



Weather Data Object

WeatherData
<code>getTemperature()</code> <code>getHumidity()</code> <code>getPressure()</code> <code>measurementChanged()</code> <code>//other methods</code>

Weather Data Object

```
1  public class WeatherData {  
2      // Declare instance variable  
3  
4      public void measurementChanged() {  
5  
6  
7  
8  
9  
10  
11  
12  
13  
14  }  
15  //other methods  
16  }
```

This method gets called when
weather data is updated

Weather Data Object

```
1 public class WeatherData {  
2     // Declare instance variable  
3  
4     public void measurementChanged() {  
5  
6  
7  
8  
9  
10  
11  
12  
13  
14 }  
15 //other methods  
16 }
```

This method gets called when
weather data is updated

Your code goes here

Weather Data Object

```
1  public class WeatherData {  
2      // Declare instance variable  
3  
4      public void measurementChanged() {  
5  
6          float temp = getTemperature();  
7          float humidity = getHumidity();  
8          float pressure = getPressure();  
9  
10  
11  
12  
13      }  
14      //other methods  
15  
16  }
```



Weather Data Object

```
1  public class WeatherData {  
2      // Declare instance variable  
3  
4      public void measurementChanged() {  
5  
6          float temp = getTemperature();  
7          float humidity = getHumidity();  
8          float pressure = getPressure();  
9  
10         curConditionsDisplay.update(temp, humidity, pressure);  
11         statisticsDisplay.update(temp, humidity, pressure);  
12         forecastDisplay.update(temp, humidity, pressure);  
13     }  
14  
15     //other methods  
16 }
```



Identify the aspects of the application that vary

```
1  public class WeatherData {  
2      // Declare instance variable  
3  
4      public void measurementChanged() {  
5  
6          float temp = getTemperature();  
7          float humidity = getHumidity();  
8          float pressure = getPressure();  
9  
10         curConditionsDisplay.update(temp, humidity, pressure);  
11         statisticsDisplay.update(temp, humidity, pressure);  
12         forecastDisplay.update(temp, humidity, pressure);  
13     }  
14  
15     //other methods  
16 }
```

Identify the aspects of the application that vary

```
1  public class WeatherData {  
2      // Declare instance variable  
3  
4  public void measurementChanged() {  
5  
6      float temp = getTemperature();  
7      float humidity = getHumidity();  
8      float pressure = getPressure();  
9  
10     curConditionsDisplay.update(temp, humidity, pressure);  
11     statisticsDisplay.update(temp, humidity, pressure);  
12     forecastDisplay.update(temp, humidity, pressure);  
13  
14 }  
15 //other methods
```

16 } Which part of this method can vary?

Identify the aspects of the application that vary

```
1  public class WeatherData {  
2      // Declare instance variable  
3  
4      public void measurementChanged() {  
5  
6          float temp = getTemperature();  
7          float humidity = getHumidity();  
8          float pressure = getPressure();  
9  
10         curConditionsDisplay.update(temp, humidity, pressure);  
11         statisticsDisplay.update(temp, humidity, pressure);  
12         forecastDisplay.update(temp, humidity, pressure);  
13  
14     }  
15     //other methods  
16 }
```

Not changed

Identify the aspects of the application that vary

```
1  public class WeatherData {  
2      // Declare instance variable  
3  
4  public void measurementChanged() {  
5  
6      float temp = getTemperature();  
7      float humidity = getHumidity();  
8      float pressure = getPressure();  
9  
10     curConditionsDisplay.update(temp, humidity, pressure);  
11     statisticsDisplay.update(temp, humidity, pressure);  
12     forecastDisplay.update(temp, humidity, pressure);  
13  
14 }  
15 //other methods  
16 }
```

Not changed

Likely to be changed why?

Program to interface, not to implementation!

Current implementation is not reusable

```
3
4 public void measurementChanged() {
5     float temp = getTemperature();
6     float humidity = getHumidity();
7     float pressure = getPressure();
8
9
10    curConditionsDisplay.update(temp, humidity, pressure);
11    statisticsDisplay.update(temp, humidity, pressure);
12    forecastDisplay.update(temp, humidity, pressure);
13
14 }
15 //other methods
16 }
```

Not changed

Likely to be changed



Part III

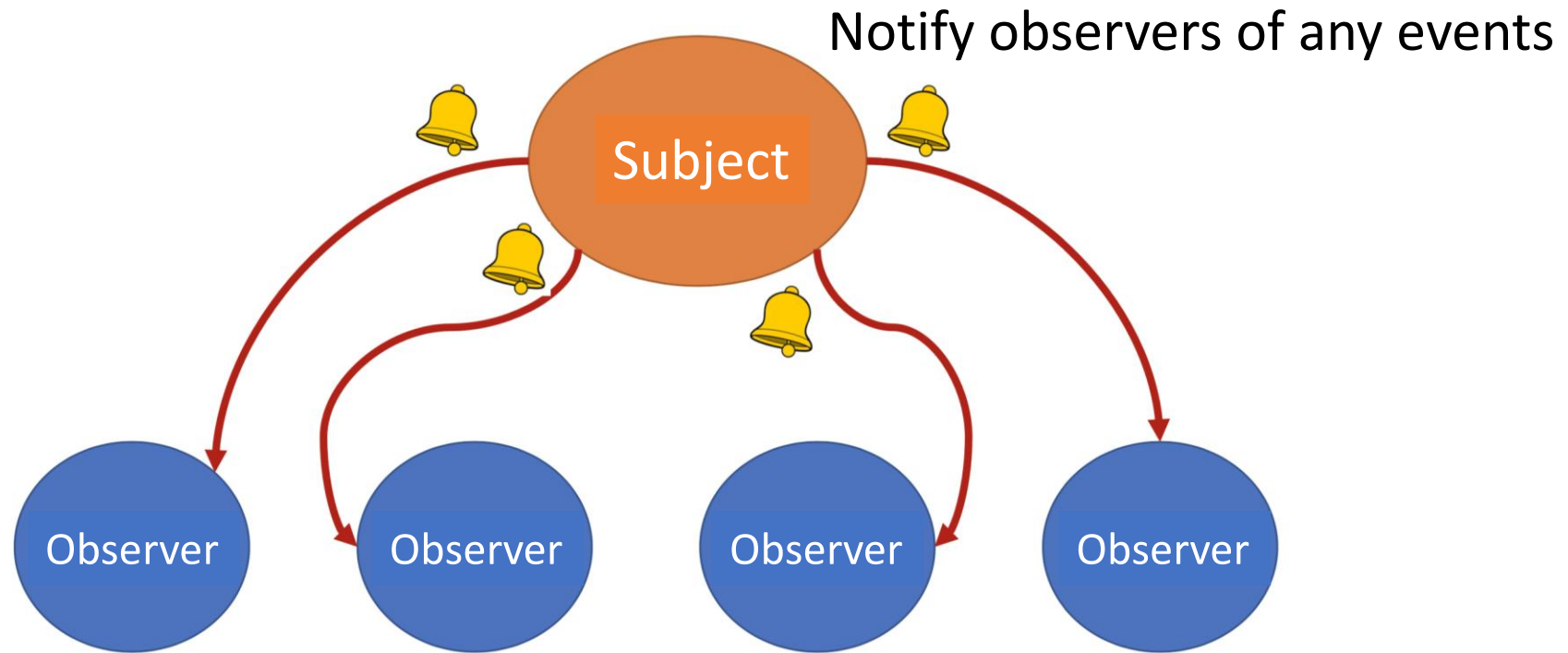
The Observer Pattern



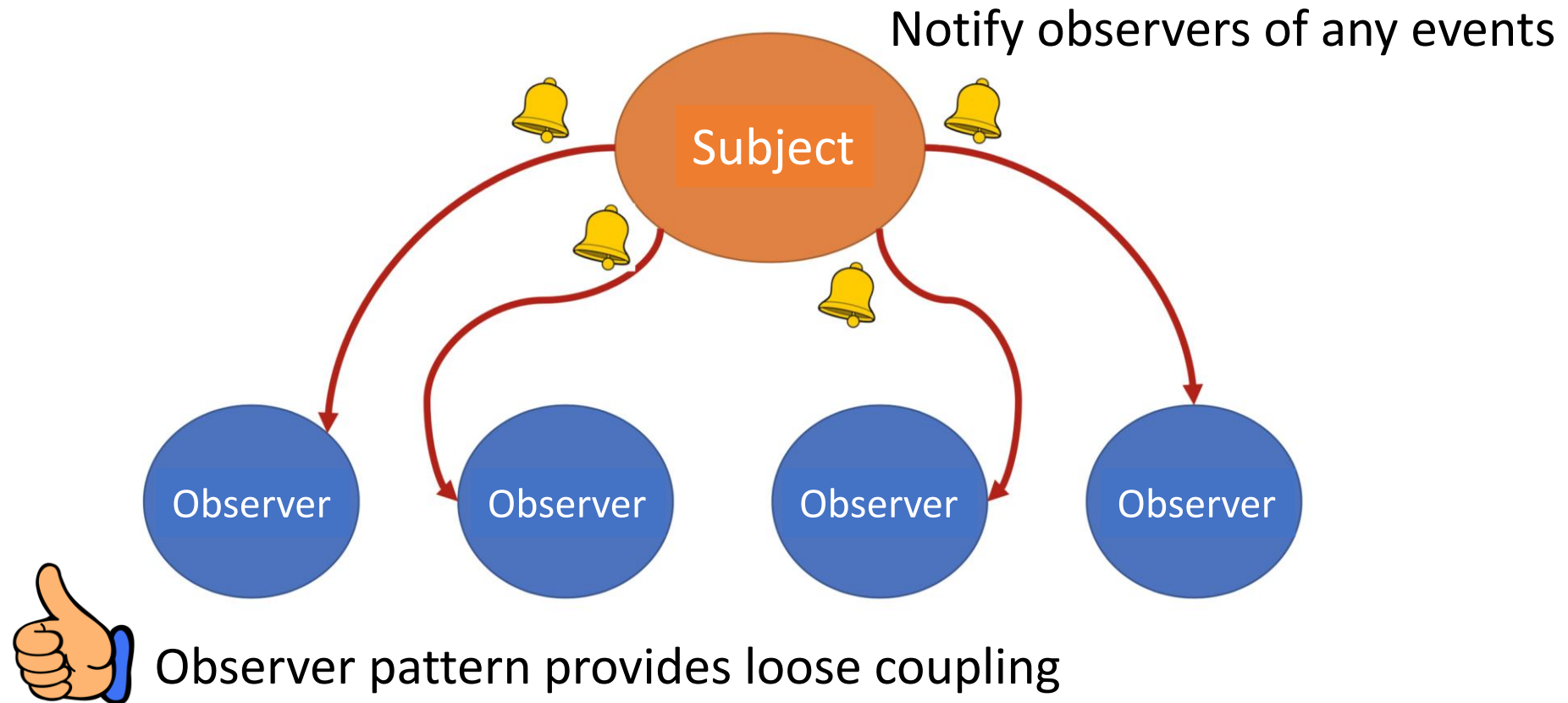
○ The Observer Pattern



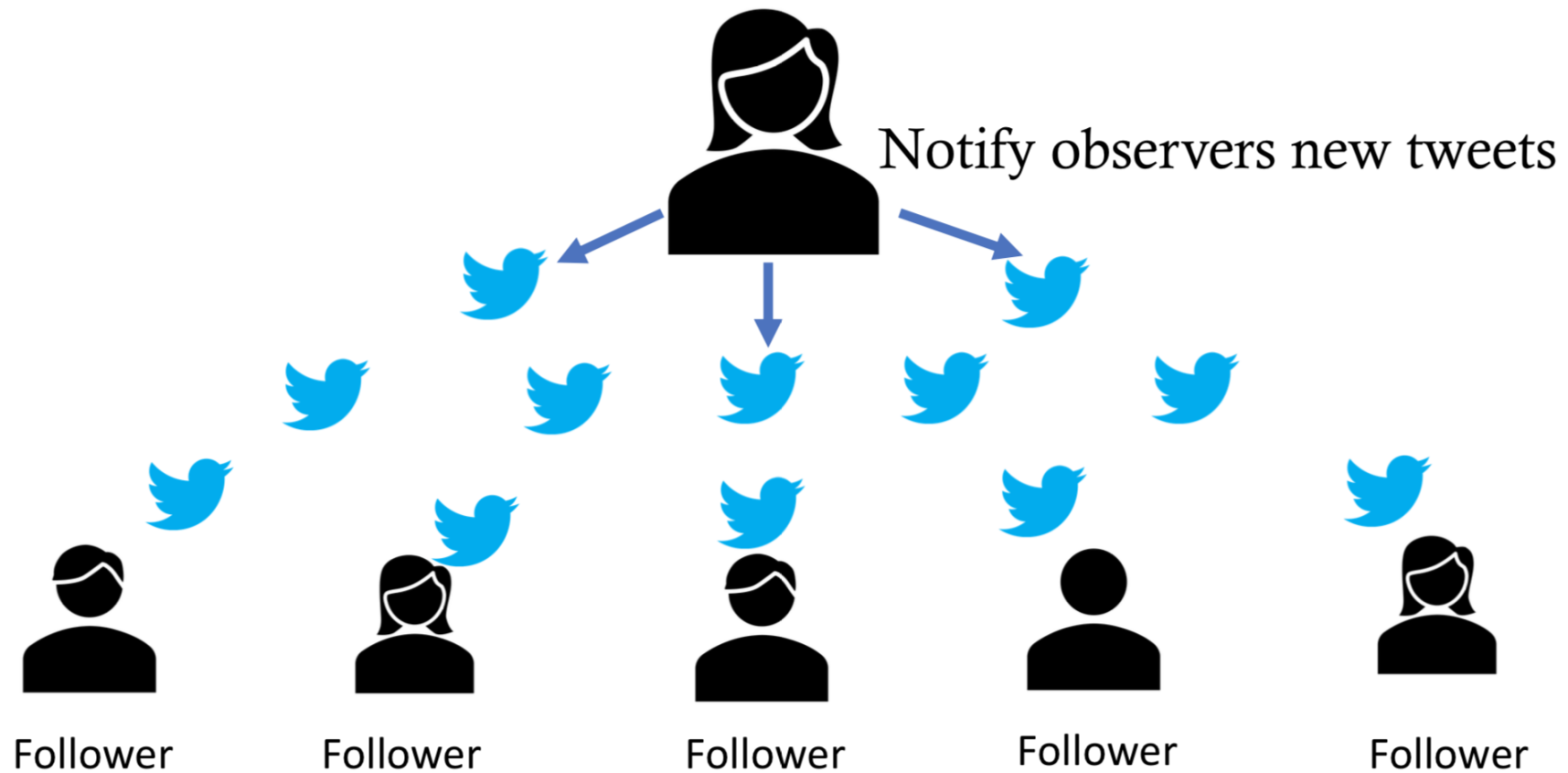
Observer Pattern



Observer Pattern



Observer Pattern



Observer Pattern

Latex Editor

Pdf Viewer

The screenshot displays a LaTeX editor on the left and a PDF viewer on the right. The editor shows LaTeX code for a document titled "An Empirical Study of Migrated Questions on Stack Exchange Websites". The code includes commands for color boxes and text colors. The PDF viewer shows the rendered document, which includes a migration notice for a question from cs.stackexchange.com. The notice states that the question is migrated from Computer Science Stack Exchange and is now available on the current site. The notice also includes the date of migration (Feb 9 '17 at 17:33) and the user who migrated it (user212926).

```
16 %\usepackage[all,defaultlines]{nowidow}
17 %\usepackage{frames/boxes}
18 \newcommand{\tastycolorbox}[1]{\begin{tcolorbox}[enhanced, left=4pt, right=4pt, top=4pt, bottom=4pt, colback=gray!5, colframe=gray!4, coltitle=white, before skip=6pt, after skip=6pt]#1\end{tcolorbox}}
19 \newcommand{\sw}[1]{\textcolor{red}{\it [Shaowei says: #1]}}
20 \newcommand{\mu}[1]{\textcolor{blue}{\it [parvez says: #1]}}
21
22 \definecolor{custom-gray}{cmyk}{0, 0, 0, 0.7, 1.00}
23 \usepackage{most}{tcolorbox}
24 \definecolor{custom-gray}{cmyk}{0, 0, 0, 0.7, 1.00}
25 \newtcbtheorem[no counter]{Summary}{\hskip-0.97em}{enhanced, drop shadow={black!50!white}, coltitle=white, top=0.15in, attach boxed title to top left={xshift=1.5em,yshift=-\tcbboxeditheight
```

An Empirical Study of Migrated Questions on Stack Exchange Websites 3

clear whether that is the case for other questions too. Stack Exchange designers would be benefitted to know whether migration solves the problem of off-topic questions or those questions need further support after migration to get faster answers or to bring more answers, edits and comments to contribute to the knowledge-base. Thus, our study is organized along the four research questions.

Store SHA-1 in database in less space than the 40 hex digits

Asked 2 years, 9 months ago Active 2 years, 9 months ago Viewed 391 times

I am using a hash algorithm to create a primary key for a database table. I use the SHA-1 algorithm which is more than fine for my purposes. The database even ships an implementation for SHA-1. The function computing the hash is returning a hex value as 40 characters. Therefore I am storing the hex characters in a `char(40)` column.

The table will have lots of rows, >= 200 Mio. rows which is why I am looking for less data intensive ways of storing the hash. 40 characters times ~200 Mio. rows will require some GB of storage... Since hex is base16 I thought I could try to store it in base 256 in hope to reduce the amount of characters needed to around 20 characters. Do you have tips or papers on implementations of compression with base 256?

data-compression database base-conversion

Click to show the list of edits in the destination site (i.e., stackoverflow.com)

share edit flag

A notice indicating that the question is migrated from Computer Science Stack Exchange website (cs.stackexchange.com)

edited Feb 10 '17 at 17:39 user212926 Last editor

asked Feb 9 '17 at 11:13 MaNa 41 3

migrated from cs.stackexchange.com Feb 9 '17 at 17:33

This question came from our site for students, researchers and practitioners of computer science.

The comment indicates that the question is programming related and not suitable in cs.stackexchange.com

(I think this is a programming question which is off-topic. The object you're trying to store is a 160-bit number)



Observer Pattern

Notify any changes
to observers

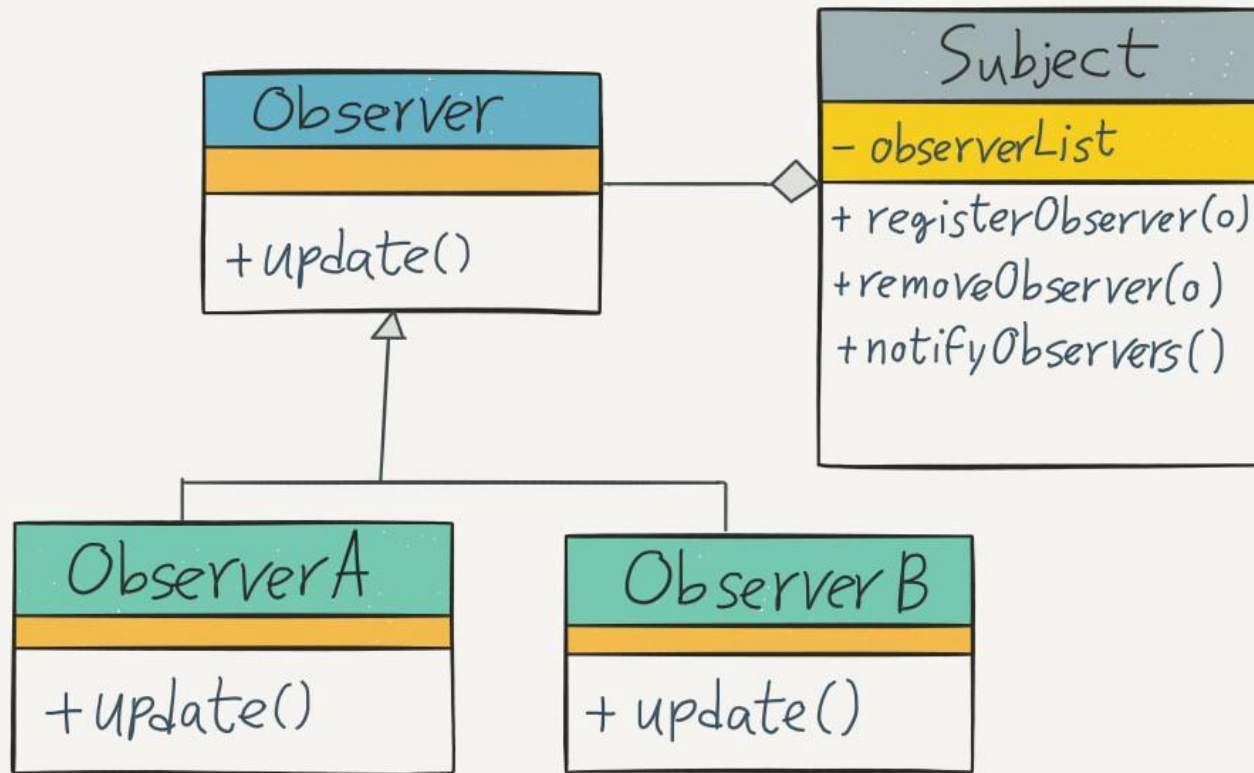
Observer

The screenshot displays a LaTeX editor interface with a file explorer on the left, a source code editor in the center, and a preview window on the right. The source code editor shows LaTeX code implementing the Observer Pattern, with line numbers 16 through 29. The preview window shows a migrated question from Stack Exchange, titled "An Empirical Study of Migrated Questions on Stack Exchange Websites". The question text discusses the use of SHA-1 for database storage and the migration of questions from Computer Science Stack Exchange to a new site. The interface includes a "Recompile" button and a "Rich Text" tab. The file explorer on the left shows a directory structure with files like "cross_cate...", "migration_cl...", "migration_e...", "migration_re...", "MigrationVo...", and "plot.migrate". The file outline on the left lists sections like "Introduction", "Background", "Data Collection", "Priliminary Study", "RQ1: Why does M...", "RQ2: Are Questio...", "RQ3: How do Use...", "RQ4: Does Migrat...", and "Implications of ou...".

```
16 %\usepackage[all,defaultlines]{nowidow}
17 %Beautiful frames/boxes
18 \newcommand{\tastycolorbox}[1]{\begin{tcolorbox}[enhanced,left=4pt,right=4pt,top=4pt,bottom=4pt,colback=gray!5,colframe=gray!4!black,before skip=6pt,after skip=6pt]#1\end{tcolorbox}}
19 \newcommand{\sw}[1]{\textcolor{red}{\it [Shaowei says: #1]}}
20 %\newcommand{\mu}[1]{\textcolor{blue}{\it [parvez says: #1]}}
21
22 \definecolor{custom-gray}{cmyk}{0, 0, 0, 0.7, 1.00}
23 \usepackage[most]{tcolorbox}
24 \definecolor{custom-gray}{cmyk}{0, 0, 0, 0.7, 1.00}
25 \newtcbtheorem[no counter]{Summary}{\hskip-0.97em}{enhanced, drop shadow={black!50!white}, coltitle=white, top=0.15in, attach boxed title to top left={xshift=1.5em,yshift=-\tcbboxeditheight
```



Observer Pattern



○ Testing the program



Using the Observer Pattern

```
1 public class WeatherData implements Subject {  
2     private ArrayList<Observer> observers;  
3     private float temperature;  
4     private float humidity;  
5     private float pressure;  
6 }
```

Implements the
Subject interface

Using the Observer Pattern

```
1 public class WeatherData implements Subject {  
2     private ArrayList<Observer> observers;  
3     private float temperature;  
4     private float humidity;  
5     private float pressure;  
6 }
```

Maintains a list of
Observers

Using the Observer Pattern

```
1 public class WeatherData implements Subject {  
2     private ArrayList<Observer> observers;  
3     private float temperature;  
4     private float humidity;  
5     private float pressure;  
6  
7     public WeatherData () {  
8         this.observers = new ArrayList<Observer>();  
9     }  
10 }
```

Create a list that will
contain the observers

Using the Observer Pattern

```
6  
7 public WeatherData () {  
8     this.observers = new ArrayList<Observers>();  
9 }  
10  
11 public void registerObserver(Observers o) {  
12     //add the observer to the list of observers  
13 }  
14 }
```

Use the method to add
an observer to the list

Using the Observer Pattern

```
6
7  public WeatherData (){
8      this.observers = new ArrayList<Observers>();
9  }
10
11 public void registerObserver (Observer o){
12     //add the observer to the list of observers
13 }
14 public void removeObserver (Observer o){
15     //remove the observer from the list
16 }
17 }
```

Use the method to remove
any observer from the list

Using the Observer Pattern

```
10 public void registerObserver (Observer o){
11     //add the observer to
12 }
13 public void removeObserver (Observer o){
14     //remove the observer from the list
15 }
16 public void notifyObservers (){
17     for (Observer observer : observers)
18         observer.update (temperature, humidity, pressure);
19 }
20 public void measurementChanged (){
21     notifyObservers();
22 }
```

Notify observers when any changes occur

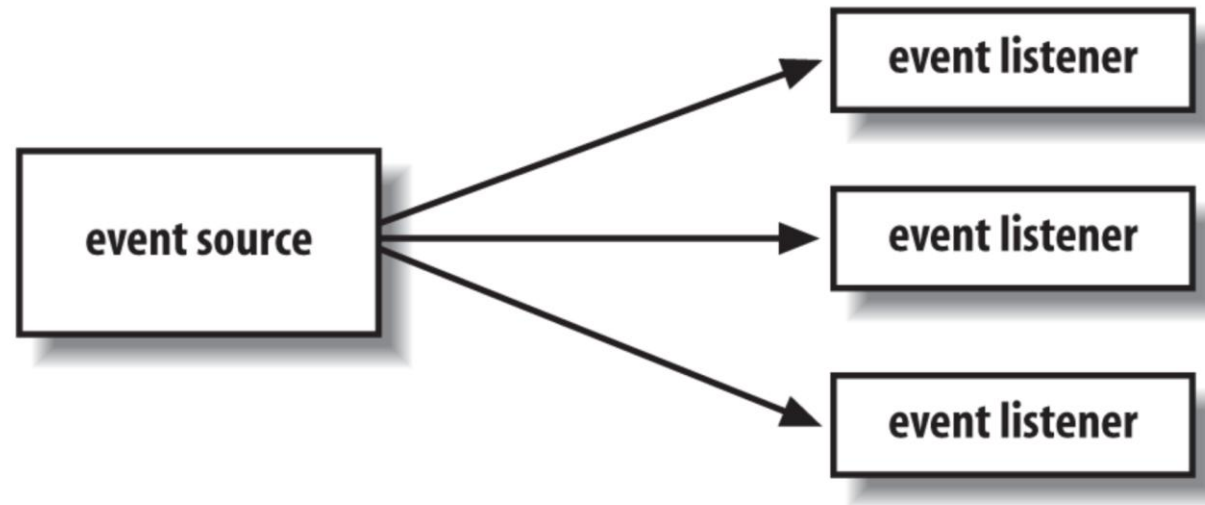


Observer Pattern Support in Java

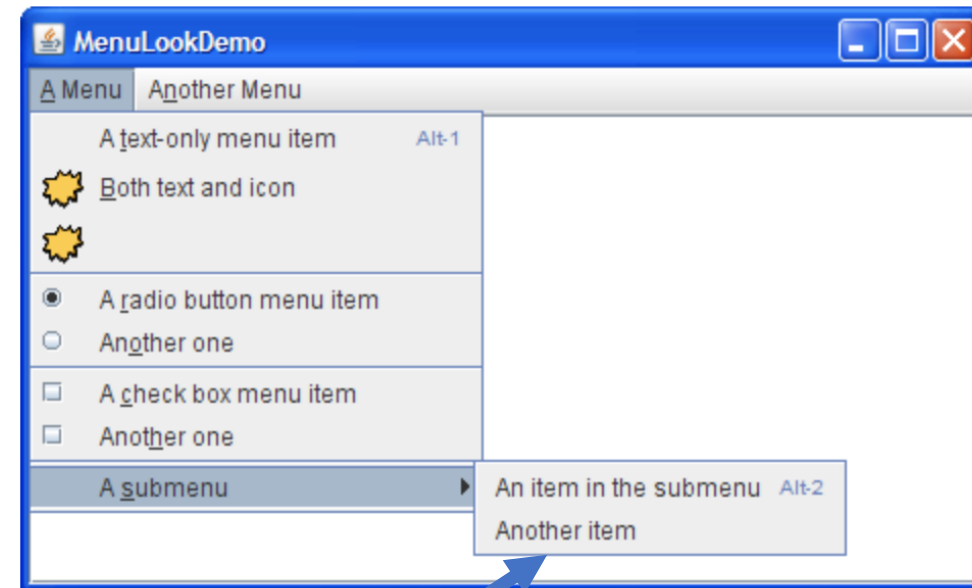
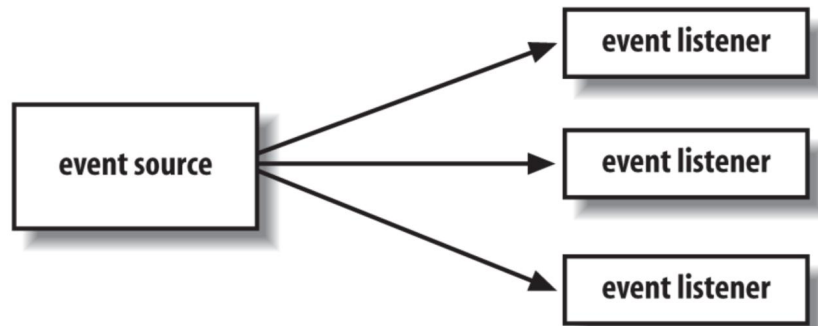
- `import java.util.observable`
- `import java.util.observer`



Examples of observer pattern in JDK

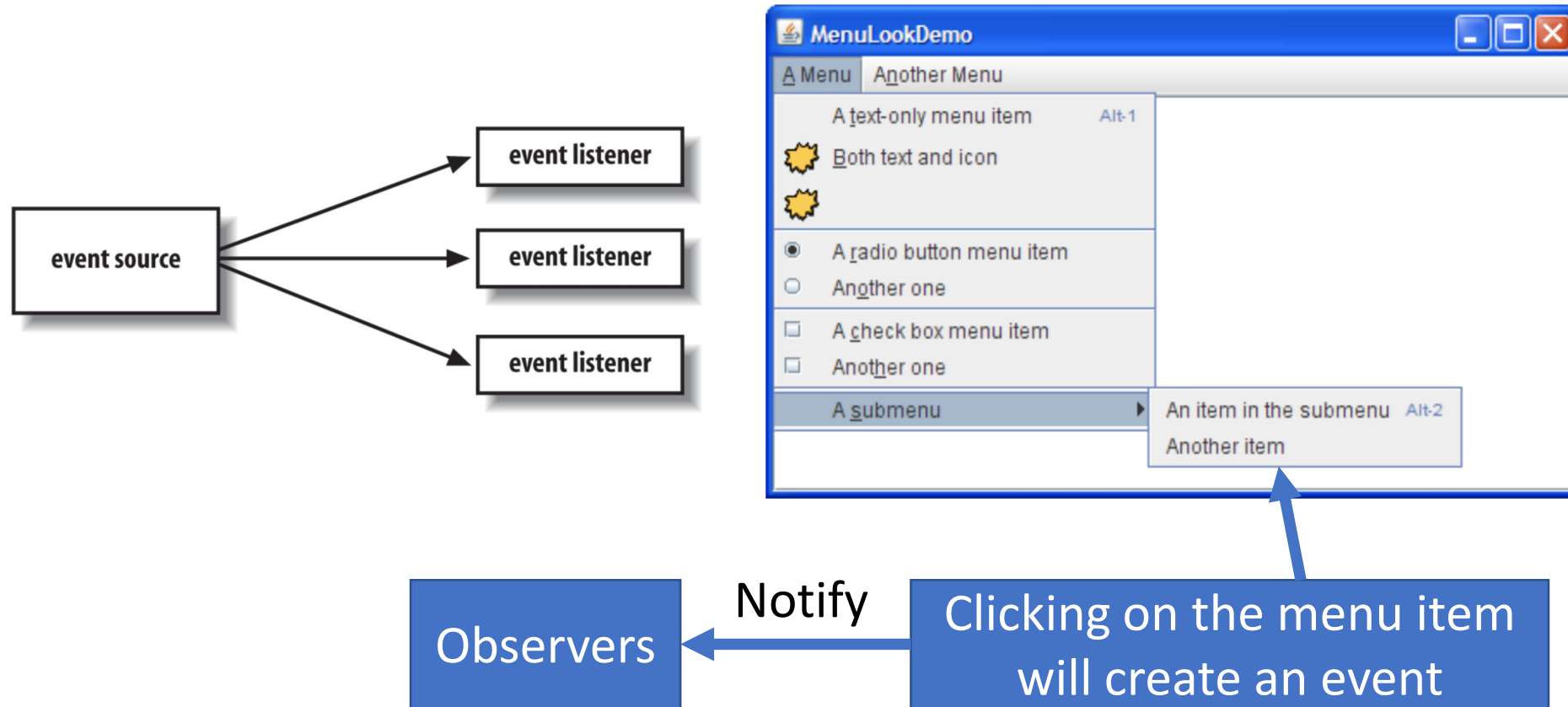


Examples of observer pattern in JDK



A menu item

Examples of observer pattern in JDK



Part IV

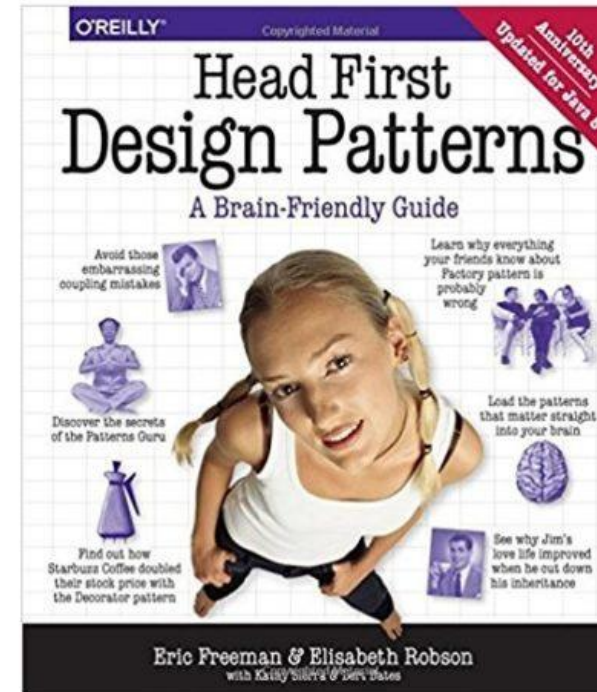
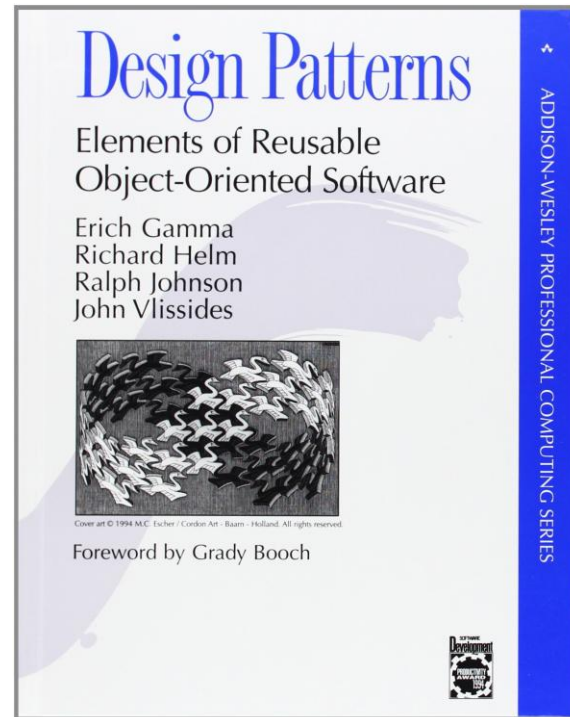
Homework, Resources and Summary



Resources



Resources



- <https://github.com/parvez2014/ObserverPatternCustomTutorial>
- <https://github.com/parvez2014/ObserverPatternJDKTutorial>

○ Homework



Homework

- Can you identify the advantages and disadvantages of using Java's observable class or the observer interface?

```
public class CurrentConditionDisplay implements Observer{  
    @Override  
    public void update(Observable o, Object arg) {  
        // TODO Auto-generated method stub  
        if(o instanceof WeatherData) {  
            WeatherData wd = (WeatherData) o;  
            System.out.println("Current conditions:" + wd.getTemperature() +  
        }  
    }  
}
```

- Can you identify any design issues in the above code? How can we fix that?

Homework

- Go through the key concepts presented in the lecture slides
- Textbook: Read the Preface
- Setup any of these tools: VS Code (with Java and GitHub extensions) | Eclipse IDE | Apache Netbeans IDE, etc., to work with Java (JDK)
- Setup GitHub as your project's repository (it is the subject of Lab #1). Do the course assigned in DataCamp:
 - [Introduction to Git – DataCamp Learn](#)
- Freshen up your knowledge on Java:
 - [Java Courses | Online Courses & Training for All Levels | DataCamp](#)
Remember to **ENROLL** to DataCamp first!
[**SUGGESTION**: Try the [Intermediate Java - DataCamp Learn](#) course]
- Be prepared to work with Java demos and examples next week



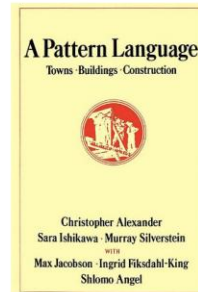
Summary



What is a design pattern?

- “Each pattern ¹describes a problem which occurs over and over again in our environment, and then ²describes the core of the solution to that problem in such a way that that you can ³use this solution a million times over, without ever doing it the same way twice”

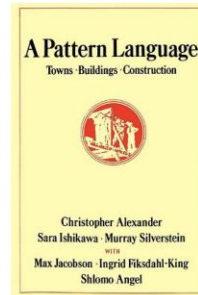
-Christopher Alexander



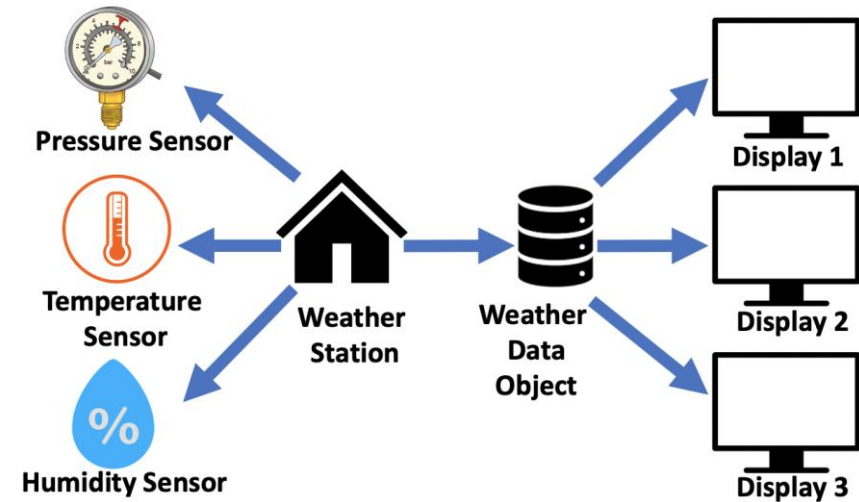
What is a design pattern?

- 1 “Each pattern **describes a problem which occurs over and over again** in our environment, and then **describes the core of the solution to that problem** in such a way that that you can
- 2
- 3 **use this solution a million times over**, without ever doing it the same way twice”

-Christopher Alexander



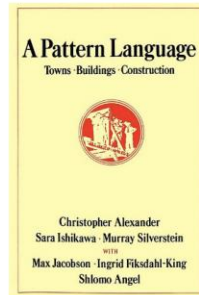
Problem Description



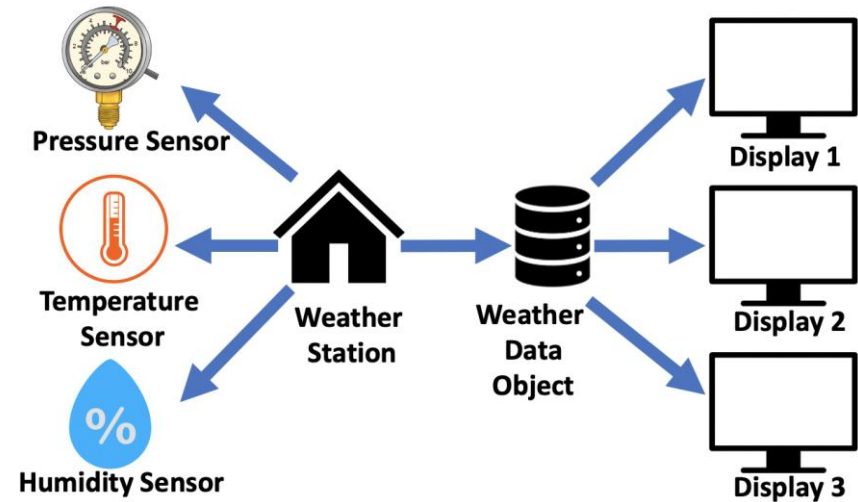
What is a design pattern?

- 1 “Each pattern **describes a problem which occurs over and over again** in our environment, and then **describes the core of the solution to that problem** in such a way that that you can
- 2
- 3 **use this solution a million times over**, without ever doing it the same way twice”

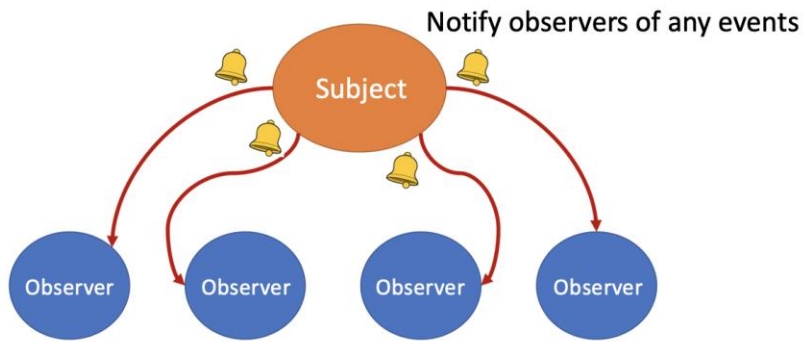
-Christopher Alexander



Problem Description



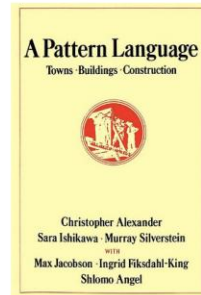
Observer Pattern



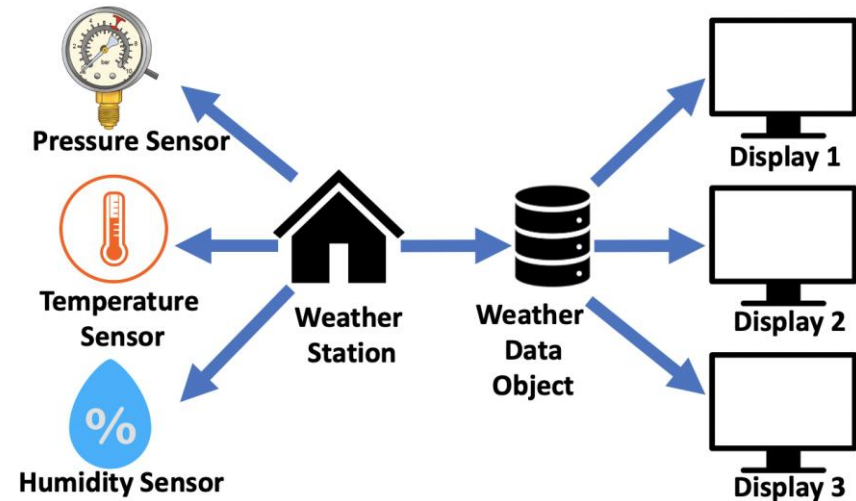
What is a design pattern?

- 1 "Each pattern **describes a problem which occurs over and over again** in our environment, and then **describes the core of the solution to that problem** in such a way that that you can
- 2
- 3 **use this solution a million times over**, without ever doing it the same way twice"

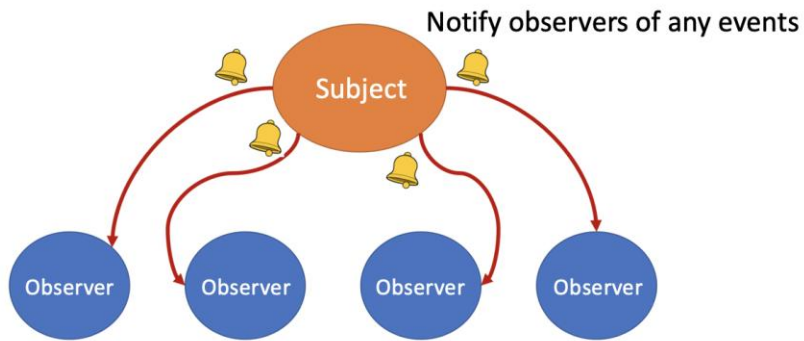
-Christopher Alexander



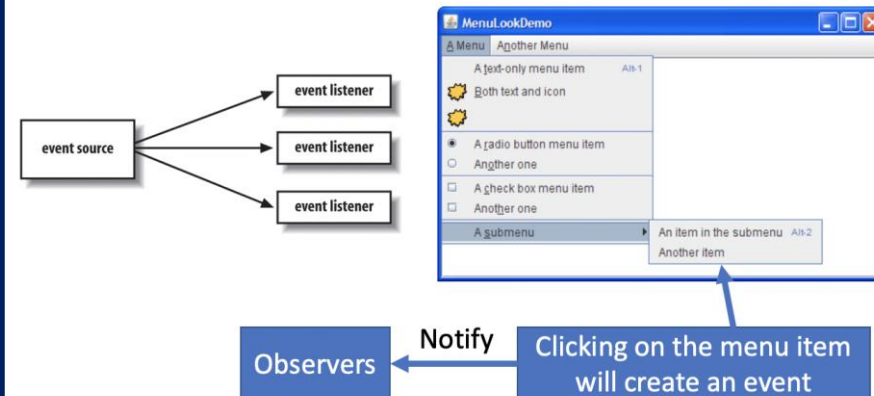
Problem Description



Observer Pattern



Examples of observer pattern in JDK



Thank you!
Questions?

Dr. Andreas S. Maniatis

Assistant Professor

Andreas.Maniatis@UWindsor.ca

<http://www.linkedin.com/in/andreasmaniatis>



University of Windsor